1 Lecture 13 (Oct 8) - Dynamic Programming: Fibonacci Numbers, Faster Text Segmentation

> **Key idea** **Dynamic programming = backtracking + bottom-up evaluation:** First get a correct recurrence (hard, creative). Then fill a table in the right order (easy, mechanical).

1.1 Fibonacci Numbers

For all integers $n \geq 0$:

$$F_n = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F_{n-1} + F_{n-2} & \text{otherwise} \end{cases}$$

Virahanka described these numbers around 800 CE. Fibonacci described them again in 1202, and his name stuck.

1.1.1 Recursive algorithm

The definition is already an algorithm:

```
RecFibo(n):
  if n = 0
    return 0
  else if n = 1
    return 1
  else
    return RecFibo(n - 1) + RecFibo(n - 2)
```

Count additions. Two recursive calls, of sizes $n-1$ and $n-2$, plus one addition:

$$T(n) = T(n-1) + T(n-2) + 1, \qquad T(0) = T(1) = 1$$

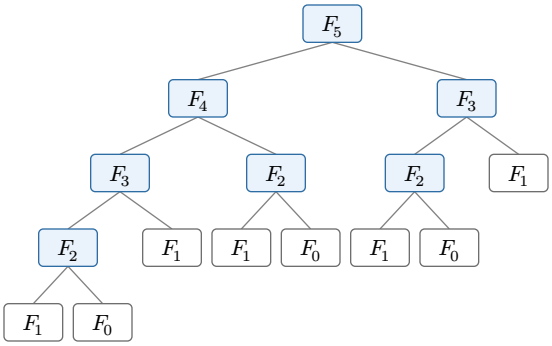
This is the Fibonacci recurrence again, plus 1. Its solution:

$$T(n) = 2F_{n+1} - 1 = \Theta(\varphi^n), \qquad \varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$$

Exponential time. You don't need to know how to solve this recurrence.

1.1.2 Why so slow?

The call tree for F_5 (each node shows the value it computes):



The same values are computed again and again:

Call	F_4	F_3	F_2	F_1	F_0
Times computed	1	2	3	5	3

The number of repeats also grows exponentially with n. Almost all of the work is wasted.

1.2 Memoization

Michie (1968) proposed **memo functions**. A memo function stores each result. When it is called again with the same input, it returns the stored result and does not compute it again. Turning a function into a memo function is called **memoization** (not a typo for "memorization": think **memo**, a written note).

How to memoize:

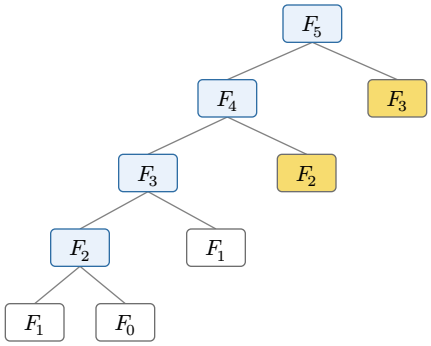
1. Keep a **global** data structure outside the function calls.
2. On each call, check whether the answer is already stored. If so, use it.
3. If not, compute it from scratch and **store it** before using it.

> **Watch out** **Use an array, not a hash table:** Array access is truly $O(1)$. Hash tables are $O(1)$ only **in expectation**, which adds a way for your running time (or your analysis) to go wrong. Usually a plain array is all you need.

```
MemFibo(i):                          ⟨⟨F[2..n] is global⟩⟩
  if i = 0
    return 0
  else if i = 1
    return 1
  else
    if F[i] is undefined
      F[i] ← MemFibo(i − 1) + MemFibo(i − 2)
    return F[i]
```

1.2.1 Analysis

The call tree for F_5 with memoization. Yellow nodes return immediately with an array lookup:

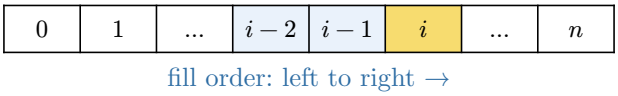


The tree is now a **path** with small leaves hanging off it. Look at what actually happens: the algorithm fills the array **in order**, $F[2], F[3], ..., F[n]$, left to right. Each entry is filled once, with one addition of two values that are already known.

$$O(n) \text{ entries} \times O(1) \text{ each} = \boxed{O(n)}$$

1.3 Iterative Fibonacci

The memoized algorithm really just fills the array from small to big. So do that **explicitly**, with a loop and no recursion:




```
IterFibo(n):
  F[0] ← 0
  F[1] ← 1
  for i ← 2 to n
    F[i] ← F[i − 1] + F[i − 2]
  return F[n]
```

One loop, so $O(n)$ additions, and easy to analyze. It is also faster in practice: no call stack, no function-call overhead.

1.4 Dynamic Programming

IterFibo is the first example of **dynamic programming**. It is more than "write a recursive algorithm and memoize it", and it is **not** "make a table and fill it in". It has two parts, and **both are required**:

Part	What	Purpose
1 · Formulate recursively	Backtracking: a recursive solution as a recurrence	**Correctness**. The hard, creative part.
2 · Build bottom-up	Fill a data structure with subproblem answers, usually iteratively	**Speed**. Mostly mechanical with practice.

Name: Bellman popularized the method in the mid-1950s while working on military scheduling problems. "Programming" means planning a schedule, as in TV programming, not writing code. "Dynamic" because the table changes over time as it fills. Today it is probably the most important technique for designing correct, efficient algorithms.

> **Watch out** **Write out every step on homework and exams:** The steps are in Erickson's *Algorithms* (chapter 3), on the back of the lab handout, and in the grading rubrics. Skipping steps costs points, and it is how incorrect algorithms happen.

1.5 Faster Text Segmentation

Recall the problem: given $A[1..n]$ and IsWord, decide whether A can be split into words. Now go through the whole process.

1.5.1 Part 1: Formulate recursively

Use backtracking to find the subproblem: a sequence of decisions (the next word), try every option, recurse on what is left.

(a) Specify the subproblem in English: Give a **closed definition** of what the function returns, like good documentation for a library function.

$$\text{Splittable}(i) = \text{True if and only if the suffix } A[i..n] \text{ can be split into words}$$

The definition may refer to the original input A. It must **not** say how to compute the value, or how the value gets used. It is a well-defined problem on its own. (Same idea as context-free grammars: you described the language of each nonterminal on its own, without the other nonterminals.)

(b) Give a recurrence: With $\text{IsWord}(i, j)$ true if and only if $A[i..j]$ is a word:

$$\text{Splittable}(i) = \begin{cases} \text{True} & \text{if } i > n \\ \bigvee_{j=i}^{n} (\text{IsWord}(i, j) \wedge \text{Splittable}(j+1)) & \text{otherwise} \end{cases}$$

Prefer a recurrence over recursive pseudocode. It makes part 2 much cleaner.

(c) Say how to answer the original problem: Call Splittable(1).

> **Key idea** Part 1 is the hard part, and part 2 means nothing without it. Everything after this point is mechanical.

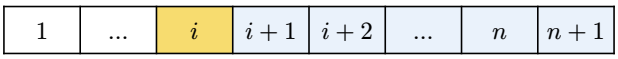
1.5.2 Part 2: Build solutions bottom-up

(a) Identify the subproblems: List every way the function can be called. Calls always move to a larger index and stop just past n. So the subproblems are Splittable(i) for all $1 \le i \le n+1$. (For Fibonacci: F_i for all $0 \le i \le n$.)

(b) Choose a memoization data structure: Usually an array, sometimes multidimensional. It must hold every subproblem: here an array SplitTable[$1..n+1$].

> **Watch out** **Pick descriptive names:** Name the array after the function it stores. Never call it `DP`, `OPT`, `ans` or `table`. The grader must be able to tell what is stored. Bad names lose points.

(c) Identify dependencies: To fill entry i, which other entries must already be known? Read it off the recurrence. Splittable(i) needs Splittable($j+1$) for every $j \ge i$: **every entry to its right**. Draw the array with arrows into entry i:



← fill order: right to left

(d) Find an evaluation order: Fill each entry only after everything it depends on. Dependencies point to **bigger** indices, so go from big to small: i from $n+1$ down to 1. Pseudocode ("for $i \leftarrow n+1$ down to 1"), words ("right to left"), or a big arrow on the drawing all count.

(e) Write the algorithm: Wrap the recurrence in loops that follow the evaluation order. Copy the recurrence as closely as possible:

```
FastSplittable(A[1..n]):
  for i ← n + 1 down to 1
    if i > n
      SplitTable[i] ← True
    else
      SplitTable[i] ← False
      for j ← i to n
        if IsWord(i, j) and SplitTable[j + 1]
          SplitTable[i] ← True
  return SplitTable[1]
```

- **No "is it undefined?" check is needed:** The evaluation order guarantees that SplitTable[$j+1$] is already filled when it is read.
- Erickson fills the base cases first, then loops over the rest. Checking the base case **inside** the loop is also fine, and copies the recurrence more directly.

(f) Analyze the running time: (Number of subproblems) × (time per subproblem). There are $n+1$ subproblems, and each tries $O(n)$ values of j. Or just read the two nested loops:

$$\boxed{O(n^2) \text{ calls to IsWord}}$$

Exponential before, quadratic now. (If IsWord took time linear in the word's length, the total would be more, perhaps $O(n^3)$.)

> **Key idea** **On homework and exams the pseudocode is optional:** The subproblem definition, recurrence, evaluation order and running time already describe the whole algorithm. Check the rubric for what is required.

1.6 Summary: the Dynamic Programming Recipe

Step	**Text segmentation**
1a. Define the subproblem in English	Splittable(i): can $A[i..n]$ be split?
1b. Recurrence (with base cases)	Big OR over the first word $A[i..j]$
1c. Which call answers the problem	Splittable(1)
2a. All subproblems	$1 \le i \le n+1$
2b. Data structure	Array SplitTable$[1..n+1]$
2c. Dependencies	Entry i needs every entry $> i$
2d. Evaluation order	i from $n+1$ down to 1
2e. Algorithm (optional)	Loops around the recurrence
2f. Running time	$O(n^2)$ calls to IsWord

Next: the lab turns five backtracking solutions for sequence problems into dynamic programming algorithms. Next week covers more backtracking patterns that lead to good dynamic programming algorithms.